

CV Sorting using LLMs: A Two-Model Pipeline for Evidence-Grounded Candidate Ranking

Capstone Project CS[02]

Abstract

Recruiters must screen hundreds of resumes per vacancy; keyword filters miss context and a single LLM prompt gives opaque scores. This project implements a terminal-based system that ranks CVs against a job description (JD) using two open-weight LLMs served through the Hugging Face Inference API. Qwen3-4B-Instruct-2507 (Model A) converts the JD into weighted, atomic requirements and converts each resume (PDF, DOCX or text) into a normalised profile. Phi-4 (Model B) then scores every requirement from 0 to 10 and must quote verbatim evidence from the resume. Quotes are verified in Python and unverifiable scores are capped. A deterministic aggregation blends the weighted requirement score, an LLM holistic score and keyword coverage, and penalises missed must-haves. The output is a ranked table plus CSV and JSON files with per-requirement explanations; recruiters can refine criteria interactively with instant re-ranking. It was run end to end on six synthetic CVs, where the ranking broadly matched the intended order of profiles; the code needs only a Hugging Face token supplied at runtime.

1. Introduction

This project combines prompt-based extraction, rubric-based scoring and rule-based verification so each rank can be audited.

Motivation. I am part of the People Tech CoreHR group at Walmart, which covers recruiting, rehiring and the employee journey, and I sit on the core interview panel, so this topic is close to my daily work. A recurring problem for technical recruiters is shortlisting the right candidate: candidate supply now exceeds demand, and manual CV screening is slow and tedious. An explainable LLM ranker removes much of that manual effort and can be strengthened with organisation-specific rules (for example, mandatory skills at a defined level) and evolved together with the product team.

2. Problem Statement

Given a folder of CVs and one job description, produce a ranked list with transparent, evidence-backed explanations using at least two LLMs, in a terminal-only Python program without embedded credentials.

3. Objectives

- Parse PDF/DOCX/TXT resumes and the JD into structured data with LLM prompts.
- Score each candidate per requirement with an overall fit score and a written explanation.
- Rank candidates, export results and let recruiters inspect evidence and refine criteria interactively.

4. Methodology

Tools. Python 3; *pypdf* for PDFs; DOCX read from its XML; the Hugging Face router (OpenAI-compatible) called via the standard library with `retry/backoff`; token from `--hf-token` or `HF_TOKEN`.

Model choice. *Model A*, *Qwen3-4B-Instruct-2507* (Apache-2.0, ungated, non-thinking): extraction needs reliable JSON and is called once per document, so a fast 4B model is cost-effective. *Qwen2.5-7B-Instruct* was not served by the enabled providers, and *Qwen3-8B* was rejected because its thinking mode risks exhausting the token budget before the JSON. *Model B*, *Phi-4* (14B, MIT, ungated): scoring needs multi-criteria reasoning, where *Phi-4* is strong for its size. Different model families for parsing and judging avoid one model grading its own extraction. *Llama-3.1-8B* and *Gemma* are gated and may block evaluators; 70B models cost too much. *pyresparser* was replaced by LLM parsing (unmaintained, spaCy/NLTK dependencies, weak on varied layouts). Resume Matcher, *LlamaIndex* and *LangChain* were not used; a small prompt-template layer keeps dependencies minimal.

Workflow and modules.

- *document_loader.py*: reads and cleans CV/JD files; skips unreadable or empty files with a warning.
- *extractor.py* (Model A): JD becomes 6-15 atomic requirements with category, weight 1-5, must-have flag and keywords; each resume becomes a profile of skills, roles, education and years, validated in Python.
- *guardrails.py*: keeps the models scoped to CV review. Every system prompt starts with a fixed CV-review scope prompt (the client refuses any other). CV, JD and requirement text is screened for injection phrases such as "you are now", "ignore the above" and "system prompt"; a match blocks the document before any model call (shown as BLOCKED). Untrusted text is fenced in markers that the prompts declare to be data.
- *matcher.py* (Model B): the prompt receives the requirements, parsed profile and resume text (name withheld to reduce bias) and returns a 0-10 score, a verbatim quote and a reason per requirement, plus a holistic score, summary, strengths and gaps. Missing requirements are re-queried once. Quotes are checked against the resume (substring or 80% token overlap): no quote caps the score at 3, an unverifiable quote at 5.
- Aggregation: $\text{final} = (0.70 \times \text{weighted requirement score} + 0.20 \times \text{LLM holistic score} + 0.10 \times \text{keyword coverage}) \times \text{penalty}$, where each missed must-have (score below 3) multiplies by 0.85, floored at 0.55. Missing components are dropped and weights renormalised.
- *ranker.py*: processes CVs in parallel threads, caches all assessments, ranks, filters and exports *ranking.csv* and *ranking.json*; *main.py* provides the CLI and interactive commands (list, show, weight, add, remove, require). Re-weighting, removal and skill filters need no LLM calls; a new requirement is scored once per CV.

5. Results and Analysis

The final code was run end to end on the Hugging Face API with six synthetic CVs (PDF, DOCX, TXT) written for a Senior Python Backend Engineer JD: strong (Alice, Fatima), partial (Bruno, Evan), junior (Chen) and off-field (Dana) profiles. The console output was:

```
$ python main.py --cv-dir ./cvs --jd ./job_description.txt --
interactive
Parsing job description with Qwen/Qwen3-4B-Instruct-2507 ...
```

```

Scoring 6 CVs for Senior Backend Engineer (Python) - Data
Platform with microsoft/phi-4 (11 requirements) ...
[done] x6 (one progress line per CV)
R1  *w5  minimum 5 years of professional experience in python
backend development
R2  *w5  strong sql and relational database skills
R3  *w4  hands-on experience deploying services on aws
R4  *w4  experience with ec2, s3, and lambda on aws
R5  *w4  experience with docker in production environments
R6  *w4  experience with kubernetes in production environments
R7   w3  experience designing rest apis
R8   w3  experience designing microservices
R9   w1  bachelor's degree in computer science or related field
R10  w3  good communication skills
R11  w3  ability to mentor junior engineers
Rank Score Reqs LLM KW% CV Missed must-haves
1      81.0  79.2   82   91  alice_nguyen.pdf -
2      73.1  71.3   75   82  fatima_rahman.docx -
3      42.7  46.2   53   73  bruno_silva.txt R6
4      38.6  38.5   56   73  evan_petrov.txt R6
5       6.6  10.8   18    9  chen_wei.txt R1 R3 R4 R5 R6
6       4.1  10.8    0    0  dana_okafor.txt R1 R2 R3 R4 R5 R6
Wrote: /tmp/cv_out/ranking.csv /tmp/cv_out/ranking.json

```

The same ranking is written to ranking.csv (adding each candidate's name, a written summary and an error column) and ranking.json (per-requirement scores and evidence quotes).

The ranking agrees with the intended order: the two Python/AWS/Kubernetes engineers lead and the off-field candidate is last; every CV received per-requirement scores, reasons and a summary. Weaknesses: Evan ranks slightly below Bruno (mainly Java) although the holistic LLM score favours Evan (56 vs 53), so per-requirement and holistic judgements can disagree; and Model A marked Kubernetes and other deployment items as must-haves although the JD labelled only Python and SQL "required". With six synthetic CVs and no human labels this is a qualitative check, not a measured accuracy. Limitations: the injection phrase filter is only a first line of defence (a CV that legitimately mentions "system prompt" is blocked for manual review), scanned PDFs are skipped, long CVs are truncated, and free Hugging Face credits do not cover a run (HTTP 402 stops cleanly; responses are cached for resuming).

6. Conclusion and Future Work

The system delivers an explainable CV ranking with two open LLMs, evidence verification, must-have penalties and interactive refinement. Future work: configurable organisation rules (mandatory skills at required levels) defined with the product team, OCR for scanned CVs, labelled evaluation against recruiter rankings, bias audits and embedding pre-filtering for large CV pools.

References

[1] Qwen Team, Qwen3 Technical Report, 2025 (Qwen/Qwen3-4B-Instruct-2507). [2] Abdin et al., Phi-4 Technical Report, Microsoft, 2024 (microsoft/phi-4). [3] Hugging Face Inference Providers docs. [4] pypdf.